

Agente Autônomo para Análise de Editais de Licitação

1. Introdução [E1]

1.1 Título

Agente Autônomo para Análise de Editais de Licitação

1.2 Equipe

Nome	E-mail	Papel principal	Contribuição principal (preencher na E2)
Danielle Magalhães Ballester	danielleballester@gmail.com	Líder de equipe / Líder de documentação / Líder de apresentação	[TODO Entrega 2]
Carol Anely Miranda Guzman	carolanely.br@gmail.com	Líder de dados	[TODO Entrega 2]
André Joás Lima de Araújo	andrejoasdev@outlook.com	Líder de engenharia / Líder de modelagem	[TODO Entrega 2]
David Damasceno da Frota	daviddf.frota@gmail.com	Líder de avaliação	[TODO Entrega 2]
Raylson Silva de Lima	raylson.ifce@gmail.com	Líder de reprodutibilidade	[TODO Entrega 2]

Tamanho da equipe: **5 integrantes**.

Papéis e responsabilidades:

- **Líder de equipe (Danielle)** — coordena planejamento, comunicação com o instrutor e divisão de tarefas.
- **Líder de dados (Carol)** — coleta, limpeza e EDA (área: Ciência de Dados).
- **Líder de modelagem (André)** — codificação, implementação do agente, pipelines de extração e parsing.

- **Líder de engenharia (André)** — infraestrutura, pipeline de dados e integração do sistema.
- **Líder de avaliação (David)** — métricas RAGAS, validação das respostas e análise de erros.
- **Líder de documentação (Danielle)** — README, relatórios e versionamento da entrega.
- **Líder de apresentação (Danielle)** — slides, narrativa e defesa no seminário.
- **Líder de reprodutibilidade (Raylson)** — ambiente, dependências, instalação e execução end-to-end.

1.3 Contexto e motivação

No ecossistema de compras públicas brasileiro, a participação em licitações exige que empresas interessadas analisem editais extensos — documentos que frequentemente ultrapassam centenas de páginas, distribuídos entre o edital principal, anexos técnicos e metadados oficiais. Informações críticas para a tomada de decisão, como prazos, modalidade, valor estimado, critérios de julgamento e exigências de habilitação, aparecem de forma fragmentada e não padronizada nesses documentos.

Atualmente, essa análise é feita manualmente por equipes jurídicas e comerciais, um processo demorado, propenso a erros por fadiga e que dificulta a triagem eficiente de oportunidades em larga escala. Embora existam soluções de busca textual, elas não oferecem extração estruturada com rastreabilidade de evidências — um requisito fundamental para auditoria e conformidade.

O avanço dos modelos de linguagem de grande escala (LLMs) combinados com técnicas de Retrieval-Augmented Generation (RAG) abre a possibilidade de automatizar essa extração com níveis de precisão e rastreabilidade que abordagens puramente baseadas em regras não alcançam. Este projeto investiga essa hipótese no contexto específico de editais de licitação do governo federal brasileiro.

1.4 Problema / pergunta de pesquisa

O problema central reside na inexistência de um sistema automatizado que extraia informações críticas de editais de licitação com precisão suficiente para apoiar a tomada de decisão empresarial, mantendo rastreabilidade completa entre cada campo extraído e sua evidência documental.

As abordagens existentes — busca textual simples, extração manual por especialistas e sistemas baseados exclusivamente em regras (regex) — apresentam limitações conhecidas: baixa escalabilidade, ausência de rastreabilidade semântica e incapacidade de lidar com a variabilidade linguística dos editais. Soluções puramente baseadas em LLMs sofrem de alucinação e falta de controle sobre as fontes.

Assim, define-se a seguinte **pergunta de pesquisa**:

Em que medida um agente autônomo baseado em RAG (Retrieval-Augmented Generation) consegue extrair campos críticos (prazo, valor estimado, modalidade) de editais de licitação com Faithfulness $\geq 0,85$ e Answer Correctness $\geq 0,80$ (métricas RAGAS), utilizando o próprio texto do edital como referência, com taxa de alucinação controlada $\leq 10\%$?

A pergunta delimita três aspectos fundamentais: (i) a abordagem técnica (agente RAG com LLM), (ii) as métricas de avaliação (RAGAS), e (iii) os critérios de sucesso (limiares quantitativos), permitindo replicação e comparação por outros pesquisadores.

1.5 Hipótese

O agente autônomo para análise de editais alcançará **Faithfulness** $\geq 0,85$ e **Answer Correctness** $\geq 0,80$ (RAGAS) nos campos críticos (prazo, valor, modalidade), utilizando o próprio texto do edital como referência, com taxa de **alucinação** $\leq 10\%$.

1.6 Objetivos

Objetivo geral: Desenvolver e avaliar um agente autônomo baseado em RAG (Retrieval-Augmented Generation) para extração estruturada de informações críticas de editais de licitação, mensurando sua eficácia por meio das métricas RAGAS (Faithfulness e Answer Correctness) tendo o próprio texto do edital como referência.

Objetivos específicos:

1. Coletar uma amostra de editais públicos e seus metadados a partir das APIs oficiais do Compras.gov.br/PNCP, abrangendo diferentes modalidades e órgãos.
2. Construir um benchmark piloto incremental (iniciando com 10 editais, adicionando até 20 conforme estabilização do desvio padrão das métricas), utilizando o texto integral dos documentos como ground truth para validação das extrações.
3. Implementar um pipeline de extração documental com suporte a OCR (Tesseract), chunking semântico e indexação vetorial (FAISS) para recuperação de trechos relevantes.
4. Implementar um agente autônomo (single-agent) com orquestração de ferramentas via LlamaIndex, incluindo busca semântica, leitura de trechos e extração estruturada com function calling.
5. Avaliar o agente utilizando as métricas RAGAS — Faithfulness (proporção de afirmações inferíveis dos chunks recuperados) e Answer Correctness (correspondência semântica com o edital) — reportando médias por campo e por edital.
6. Mensurar custo operacional (tokens, chamadas de API) e latência por edital como indicadores de viabilidade prática.
7. Documentar o pipeline de forma reproduzível, com dependências fixadas, instruções de instalação e roteiro de execução end-to-end.

2. Dados

2.1 Fonte e licença [E1]

A fonte principal será o ecossistema de compras públicas do Governo Federal, incluindo:

- **Portal Nacional de Contratações Públicas (PNCP)** — APIs públicas com metadados das contratações e documentos vinculados.
- **Compras.gov.br** — Portal de Compras do Governo Federal.
- **Licença:** Dados públicos e abertos (Lei de Acesso à Informação — Lei 12.527/2011).
- **Data de acesso:** De 09 de Junho de 2026 até 15 de Junho de 2026.
- **Autenticação:** APIs públicas não requerem autenticação para consulta.

2.2 Volume e formato [E1]

Item	Descrição
Nº de amostras (piloto)	10–20 editais (incremental, conforme estabilização do desvio padrão)
Formato	JSON/XML para metadados; PDF para edital e anexos
Campos-alvo	Modalidade, valor estimado, prazos, critério de julgamento, objeto, requisitos de habilitação
Tamanho estimado	Variável por edital (dezenas de KB a centenas de MB com anexos)
Periodicidade	Snapshot estático para o benchmark piloto

2.3 Campos de extração [E1]

Campo	Tipo	Descrição
prazo_entrega_proposta	Data	Data limite para entrega de propostas
valor_estimado	Monetário	Valor estimado da contratação

modalidade	Categórico	Pregão, concorrência, dispensa, inexigibilidade
objeto	Texto	Descrição do objeto da licitação
orgao_responsavel	Texto	Órgão/entidade licitante
uf	Categórico	Unidade federativa do órgão
criterio_julgamento	Categórico	Menor preço, técnica e preço etc.
prazo_execucao	Data	Prazo de execução do contrato

Os três primeiros são **campos críticos** avaliados com RAGAS (Faithfulness + Answer Correctness). Os demais são **descritivos**, extraídos sem meta quantitativa.

2.4 Riscos de dados [E1]

Risco	Impacto	Tratamento planejado
PDF escaneado (sem texto extraível)	Alto — impede extração direta	Fallback para OCR via Tesseract
Anexos extensos	Médio — aumenta custo/latência	Chunking seletivo por seção
Orçamento sigiloso	Médio — campo ausente	Tratar como indisponibilidade, não como erro do modelo
Campos ausentes em alguns editais	Baixo — esperado	Schema flexível com campo de confiança
Amostra homogênea (pouca variedade)	Baixo — amostral	Coleta diversificada por órgão, UF e modalidade

2.5 Pré-processamento aplicado [E2]

[TODO Entrega 2] Testar as quantidades de chunks, adicionar overlap, melhorar a busca semântica. Compare com o que foi previsto em 2.4.

2.6 Ética e privacidade [E1, revisar na E2]

Os dados utilizados são públicos, provenientes de portais oficiais do governo federal (PNCP, Compras.gov.br), sem conter informações pessoais (PII). A licença é aberta, permitindo uso acadêmico. O sistema deve ser usado como apoio à triagem, não como substituto de análise jurídica ou profissional. O projeto não envolve dados sensíveis nem requer anonimização.

3. Metodologia

3.1 Abordagem [E1]

A abordagem adotada combina **extração documental**, **indexação semântica** e **agente autônomo com LLM** para extrair informações estruturadas de editais de licitação. O sistema é composto por três camadas:

1. **Extração e preparação dos documentos** — texto extraído via PyMuPDF com fallback OCR (Tesseract) para PDFs escaneados, segmentado em chunks de tamanho fixo.
2. **Indexação vetorial** — cada chunk é convertido em embeddings via `sentence-transformers/all-MiniLM-L6-v2` e indexado no FAISS (CPU) para recuperação por similaridade semântica.
3. **Agente de extração** — um agente LlamaIndex (`FunctionAgent`) orquestra ferramentas de busca semântica e geração estruturada via função de prompt com Groq API (`openai/gpt-oss-120b`), retornando respostas acompanhadas de evidências textuais.

A avaliação utiliza o próprio texto do edital como ground truth, comparado à resposta do agente por meio das métricas RAGAS.

3.2 Stack técnica [E1]

Componente	Tecnologia
Linguagem	Python ≥ 3.10
Framework web	Flask
LLM	Groq API (openai/gpt-oss-120b)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Vector store	FAISS (CPU)
Framework de agentes	LlamaIndex (FunctionAgent)
Extração PDF	PyMuPDF (fitz)
OCR (fallback)	Tesseract + pytesseract + Pillow
Gerenciamento de dependências	uv (ou pip + requirements.txt)

Versionamento	Git
---------------	-----

3.3 Baseline [E1]

A avaliação usa o **próprio texto do edital como referência (ground truth)** via métricas RAGAS:

Componente	Descrição
Referência	O texto integral do edital (ground truth)
Métricas RAGAS	Faithfulness e Answer Correctness
Função	Mensurar se a resposta do agente é inferível do edital e semanticamente correta

Essa abordagem substitui as baselines tradicionais porque o edital é a fonte mais autoritativa para validar a extração — elimina ambiguidade externa e permite medir qualidade semântica, não apenas acerto exato de padrão.

3.4 Pipeline [E2]

O pipeline implementado no código-fonte (`app.py`) segue o fluxo abaixo:

```
PDF (upload) → Extração de texto (PyMuPDF + OCR fallback)
→ Chunking (500 palavras)
→ Embeddings (all-MiniLM-L6-v2)
→ Indexação FAISS
→ Consulta do usuário → busca semântica (top-5 chunks)
→ Prompt com contexto + LLM (Groq)
→ Resposta estruturada com evidências
```

Cada etapa é executada no momento do upload (indexação) e no momento da consulta (recuperação + geração). O índice FAISS é persistido em disco (`vector_index.faiss`) e atualizado incrementalmente a cada novo documento enviado.

3.5 Modelos comparados [E2]

[TODO Entrega 2] Único modelo implementado até o momento: agente LlamaIndex com Groq. Prevê-se experimentação com diferentes LLMs, estratégias de chunking e temperaturas.

3.6 Protocolo de validação [E1]

Item	Definição
Método de avaliação	RAGAS (Faithfulness + Answer Correctness) comparando resposta do agente contra o texto do edital
Conjunto de teste	10–20 editais (incremental: lote inicial de 10, +5 a cada rodada se desvio padrão > 0,10; cap de 20)
Componentes RAGAS	Faithfulness (afirmações inferíveis dos chunks recuperados), Answer Correctness (correspondência semântica com o edital)

3.7 Métricas e critérios de sucesso [E1: planejados; E2: aplicados]

Métrica (RAGAS)	Definição	Meta inicial (mínima)
Faithfulness	Proporção de afirmações na resposta que são inferíveis dos chunks recuperados	$\geq 0,85$
Answer Correctness	Similaridade semântica entre a resposta gerada e o texto do edital como referência	$\geq 0,80$
Custo por edital	Tokens, chamadas de API e tempo por documento	Reportar média
Latência	Tempo total de processamento por edital	Reportar média/mediana

4. Cronograma [E1: planejado; E2: atualizado]

A janela real de implementação é de ~9 semanas (29/mai → 31/jul/2026), com 2 marcos formais.

Na **Entrega 2**, atualize a coluna **Status** marcando cada semana como [OK], [Atrasado] ou [Não feito], e adicione 1 linha com aprendizado sobre desvios.

Semana	Período	Marco / atividade prevista	Status (E2)
1	29/mai – 05/jun	Pré-especificação + aquisição de dados + EDA inicial	[TODO E2]
2	06–12/jun	Definição do schema de campos-alvo + início da anotação do benchmark	[TODO E2]
3	13–19/jun	Pipeline de extração de texto + OCR + chunking	[TODO E2]
4	20–26/jun	Protótipo single-agent com LangGraph/LlamaIndex + extração estruturada	[TODO E2]
5	27/jun – 03/jul	Entrega 1 (30/jun) + refinamento pós-feedback	[TODO E2]
6	04–10/jul	Integração RAGAS + tuning do agente + experimentos multi-agent	[TODO E2]
7	11–17/jul	Métricas RAGAS (Faithfulness, Answer Correctness) + custo e latência	[TODO E2]

8	18–24/jul	Documentação final (preencher seções E2) + ensaio de demo (reprodutibilidade)	[TODO E2]
9	25–31/jul	Entrega 2 (31/jul) — Seminário + slides + defesa	[TODO E2]

Os marcos das semanas 5 e 9 são fixos; as demais atividades podem ser realocadas conforme a complexidade.

Aprendizado sobre desvios (E2): A avaliação passou a usar RAGAS (Faithfulness + Answer Correctness) com o próprio edital como ground truth, eliminando a necessidade dessas baselines como comparativo.

5. Resultados [E2]

5.1 Métricas obtidas

[Resultados a serem preenchidos na Entrega 2 após execução do pipeline de avaliação.]

Métrica (RAGAS)	prazo_entrega_propria	valor_estimado	modalidade	Global (média)
Faithfulness	[TODO]	[TODO]	[TODO]	[TODO]
Answer Correctness	[TODO]	[TODO]	[TODO]	[TODO]

Indicador	Valor
Custo médio/edital	[TODO tokens/\$\$]
Latência média	[TODO segundos]
Latência mediana	[TODO segundos]

5.2 Gráficos relevantes

[TODO Entrega 2] Matriz de confusão, curva PR, curva ROC, SHAP etc. Inclua ou referencie imagens em docs/figs/.

5.3 Análise de erros

[TODO Entrega 2] Onde o modelo falha? Casos sistemáticos de falso positivo/negativo?

5.4 Comparação com a hipótese

[TODO Entrega 2] A hipótese declarada em 1.5 foi confirmada? Parcialmente? Explique com evidências.

6. Conclusão [E2]

6.1 Principais achados

[TODO Entrega 2] 3-5 pontos concretos que o projeto demonstrou.

6.2 Limitações

[TODO Entrega 2] O que não foi feito, o que não generaliza, onde o projeto não deveria ser aplicado.

6.3 Trabalhos futuros

[TODO Entrega 2] O que você faria com mais 6 semanas?

6.4 Aprendizados da equipe

[TODO Entrega 2] O que a equipe aprendeu (técnico e de processo)?

7. Reprodutibilidade [E2]

7.1 Requisitos

- Python: [TODO Entrega 2 versão, ex.: 3.11+]
- [TODO Entrega 2 lista de libs principais ou referência ao pyproject.toml]
- Sistema operacional testado: [TODO Entrega 2]
- Semente global: [TODO Entrega 2, ex.: 42]

7.2 Instalação

```
# Clonar o repositório
git clone [TODO URL]
cd [TODO nome-do-projeto]

# Instalar dependências com uv (recomendado)
uv sync

# Alternativa com pip
# pip install -r requirements.txt
```

7.3 Obter os dados

[TODO Entrega 2: comando de download ou instrução para baixar manualmente]

7.4 Executar o pipeline

```
# Ex.:
# python app.py
```

7.5 Rodar os testes (se aplicável)

```
# Ex.: pytest -v
```

7.6 Artefatos gerados

[TODO Entrega 2] Liste o que será gerado em artifacts/, models/, reports/. Tamanho esperado.